## Integer and Floating-Point Numbers

Ruby integers are objects of class Integer. Integer objects hold integers that fit within the native machine word minus 1 bit. The internal storage of integers changes depending on the size of the number, but for nearly every purpose in Ruby, that internal detail is abstracted away from you.

Integers are written using an optional leading sign and an optional base indicator (0 or 0o for octal, 0d for decimal, 0x for hex, or 0b for binary), followed by a string of digits in the appropriate base. Underscore characters are ignored in the digit string.

```
123456                  => 123456
0d123456                => 123456
123_456                 => 123456   # - underscore ignored
-543                    => -543     # - negative number
0xaabb                  => 43707    # - hexadecimal
0377                    => 255      # - octal
0o377                   => 255      # - octal
-0b10_1010              => -42      # - binary (negated)
123_456_789_123_456_789 => 123456789123456789 # Big number internal storage
```

A numeric literal with a decimal point and/or an exponent is turned into a Float object, corresponding to the native architecture's double data type. You must follow the decimal point with a digit; if you write 1.e3, Ruby tries to invoke the method e3 on the Fixnum 1. You must place at least one digit before the decimal point.

```
12.34     # => 12.34
-0.1234e2 # => -12.34
1234e-2   # => 12.34
```

### Rational and Complex Numbers

Classes that support rational numbers (ratios of integers) and complex numbers are built into the Ruby interpreter. Rational numbers have a literal syntax with an expression followed by the letter r:

```
2r        # => (2/1)
3/4r      # => (3/4)
-2/5r     # => (-2/5)
2/-5r     # => (-2/5)
1.4r      # => (7/5)
3/9r      # => (1/3)
```

Complex numbers are integers with an i suffix for the complex part:

```
2i        # => (0+2i)
4 + 3i    # => (4+3i)
```

A number can be rational and imaginary, but the r needs to come before the i in the suffix list.

## Strings

Ruby provides a number of mechanisms for creating literal strings. Each generates objects of type String. The different mechanisms vary in terms of how a string is delimited and how